Process & Decision Document

1. Short Description

For this Side Quest, I created a branching interactive story using p5.js that unfolds across multiple game states and files.
The project focuses on decision-making and consequence by allowing the player to navigate a small narrative tree where each choice affects a tracked stat called **KARMA**. Rather than emphasizing action or difficulty, the experience is designed around reflection, cause-and-effect awareness, and replayability.

The structure separates functionality into different files (start screen, game state, win/lose outcomes, and instructions) while a central router manages transitions between them. The story content itself is organized into a data-driven object that defines narrative nodes and branching paths.

The main technical focus was:

- Building a multi-state system across separate JavaScript files
- Designing a small branching decision tree structure
- Tracking a persistent player stat across scenes
- Resolving multiple endings based on accumulated choices
- Creating accessible interaction through both mouse and keyboard input

2. One Small Decision I Made

Originally, I considered writing the story flow using a series of conditional `if/else` statements to control each branch.

However, this quickly became difficult to manage and would not scale well if additional scenes were added.

Instead, I reorganized the narrative into a centralized **STORY object** that stores each scene as structured data (text, choices, stat changes, and next node). This allowed the game logic to remain simple while making the narrative easier to expand or modify.

This decision:

- Made the code more modular and readable

- Allowed branching to function like a lightweight narrative database
- Reduced repetition in the logic controlling transitions
- Supported experimentation with additional paths without restructuring the system

3. GenAI Use Statement

GenAI was used to assist with:

- Structuring the multi-file game state system and screen routing
- Debugging transitions between story nodes and endings
- Refining how the KARMA stat persists and resolves different outcomes
- Providing suggestions for organizing branching narrative data

All generated suggestions were reviewed, tested, and adjusted.
The final parameters, layout decisions, and visual behavior were iterated manually during development.

No external assets were generated or imported; all visuals are drawn procedurally in code.

Appendix: GenAI Transcript

ChatGPT 5.2 Share

main.js
JavaScript

game.js
JavaScript

start.js
JavaScript

win.js
JavaScript

Provide suggestions for organizing branching narrative data

Here are **practical ways to organize branching narrative data** based on how your current project is structured. I'll reference your existing implementation so you can evolve it instead of rewriting everything.

✅ 1. Use a "Node-Based Story Graph" (You Already Started This — Good!)

Right now your narrative is stored as a dictionary-style object called `STORY`, where each key is a node ID (`intro`, `cafe`, etc.) and each node defines text + choices. game

This is actually the **correct foundation** for branching narratives because it behaves like a graph:

ChatGPT 5.2 Share

sketch.js
JavaScript

This is my code and can you check about structuring the large scrolling world system

Thought for 5s

I looked through your `sketch.js` and your "large scrolling world" structure is already on the right track: you're doing **world-space updates**, then drawing with `translate(-cam.x, -cam.y)`, then doing **screen-space HUD**. That's the standard, scalable pattern for big maps. sketch

Below are the main structural issues + upgrades I'd make so this stays clean and performant as the world gets larger.

---

1) Bug: `updateRipples()` is defined twice (the first one is overwritten)

You have **two functions named** `updateRipples(t)`. In JavaScript, the later one replaces the earlier one, so the earlier version is effectively dead code. sketch

Fix: keep only one (the later one is better since it su…rts `world` vs `screen` ripples). Delete the earlier `updateRipples`.

Ask anything